Beyond Bit-Width: SIMD Dequantization Overhead Creates a CPU/GPU Performance Divide in GGUF K-Quant LLM Inference

Kris D'Costa
Halicioglu Data Science Institute
University of California, San Diego
San Diego, CA, USA
kdcosta@ucsd.edu

***Abstract*—Deploying quantized LLMs on CPU requires choosing among GGUF K-quant variants (Q2_K through Q8_0), but the dominant design intuition—higher bit-width yields higher throughput—is *wrong* on CPU. We present the first mechanistic, cross-platform study explaining why. On ARM (Pixel 6a, Cortex-X1) and x86 (Intel i5-1235U, AVX2), decode throughput is *non-monotonic* with bit-width: Q2_K leads at 5.11 tok/s (ARM) and 14.1 tok/s (x86), while Q6_K is slowest (3.52 and 6.8 tok/s respectively)— *regardless of platform*. Direct inspection of llama.cpp's NEON and AVX2 kernels explains the mechanism: Q6_K stores weights split across two arrays (`ql[128] + qh[64]`), requiring ≈26 NEON operations per 256-weight superblock to reconstruct 6-bit values—nearly 3× the instruction count and data bandwidth of Q2_K's simple 2-bit masking (≈10 ops, 84 bytes/block). This SIMD dequantization overhead creates a CPU/GPU divide: only Apple Metal—with dedicated 4-bit kernel paths—reverses the ordering (Q4_K_S leads at 19.9 tok/s; Q8_0 is slowest at 6.4 tok/s). We further discover that KV-cache context sensitivity is also non-monotonic: Q2_K degrades -48% on ARM (cliff at ctx=512, matching the $L2/1024$ formula exactly) and -51% on x86 (cliff at ctx=1200–1300), while Q3_K_M varies $< \pm 11\%$ on both. We derive a cache-overflow formula—per-layer K+V footprint = $2048 \times$ ctx bytes—that predicts the x86 cliff threshold within 5% of the observed value from the i5-1235U's 1.25 MB L2 cache capacity alone. Finally, we show that WikiText-2 perplexity is monotonic with bit-width yet fails to predict task accuracy: Q5_K_M through Q8_0 differ by only 0.035 PPL nats but span 10+ percentage-point accuracy gaps on BoolQ and TruthfulQA. All 700+ measurements, benchmark scripts, and figures are publicly available.**

***Index Terms*—LLM inference, quantization, GGUF, llama.cpp, ARM, x86, SIMD, KV-cache, mobile inference, edge AI, CPU/GPU divide, NEON, AVX2**

I. Introduction

A. The CPU Inference Problem

The rapid deployment of LLMs on CPU-only endpoints—mobile phones, laptops, edge servers—has converged on a de facto standard: GGUF K-quant formats implemented in llama.cpp, powering Ollama, LLM Farm, and millions of on-device inferences daily. Practitioners choose among seven variants (Q2_K through Q8_0, spanning 2.6–8.5 bits per weight) guided by a single heuristic borrowed from GPU literature: *higher bit-width yields higher throughput and quality*. This paper shows that heuristic is wrong on CPU, explains mechanistically why, and provides validated cross-platform evidence on two CPU ISAs and one GPU.

The GPU intuition fails because it ignores **SIMD dequantization overhead**. On GPU, dedicated hardware kernels (e.g., CUTLASS 4-bit GEMM on NVIDIA, Apple's ANE 4-bit matrix multiply) fuse weight loading and computation, making throughput nearly proportional to memory bandwidth—and thus to bit-width. On CPU, the NEON and AVX2 `ggml_vec_dot` kernels must *explicitly reconstruct* integer weights from their packed representations before accumulation. As we show via direct kernel inspection (llama.cpp commit `1a29907`), Q6_K requires splitting its 6-bit values across two memory arrays (`ql[128] + qh[64]` per 256-weight block) and executing eight bit-manipulation instructions per group to reconstruct each value—nearly 3× the instruction count and 3× the data bandwidth of Q2_K's simple AND-mask extraction. This overhead dominates the per-token decode budget on both ARM NEON and x86 AVX2, inverting the expected throughput ordering.

B. Key Findings at a Glance

- **Non-monotonic throughput:** Q2_K is fastest on both ARM (5.11 tok/s) and x86 (14.1 tok/s); Q6_K is slowest (3.52 and 6.8 tok/s)—the same rank ordering on both ISAs.
- **CPU/GPU divide:** Mac M4 Metal *reverses* the ordering (Q4_K_S leads at 19.9 tok/s; Q8_0 slowest at 6.4 tok/s), demonstrating that the anomaly is a CPU property, not a model property.
- **KV-cache cliff with L2-derivable thresholds:** Q2_K throughput drops -48% on ARM (cliff at ctx=512, matching $L2/1024 = 512$ KB$/1024 = 512$ tokens *exactly*) and -51% on x86 (cliff at ctx=1200–1300). Both thresholds are predicted within 5% by $L2_{\text{cache}}/1024$—the point where the per-layer combined K+V cache overflows L2.
- **Q3_K_M cliff immunity:** Q3_K_M varies $< \pm 5\%$ across all contexts on both platforms, not because its KV cache is smaller (it is identical—same fp16 format, same

architecture), but because its higher FFN compute cost makes the relative attention overhead smaller.

- **PPL $\neq$ task accuracy:** WikiText-2 perplexity is monotonically decreasing with bit-width, yet task accuracy is not. Q5_K_M through Q8_0 differ by 0.035 PPL nats but span 10+ percentage-point gaps on BoolQ and TruthfulQA. Perplexity is an insufficient proxy for deployment selection.

C. Contributions

C1: Empirical characterization Empirical characterization of 70+ m 2.3B Instruct measurements) measured on ARM (Pixel 6a, Cortex-X1), x86 (Intel i5-1235U, AVX2), and Mac M4 Metal across decode TPS, prefill TPS, context-scaling (11 context sizes, filled-context methodology), 6 quality benchmarks (n=100 each), and full-corpus WikiText-2 perplexity (568 chunks, 285K tokens).

C2: Mechanistic kernel analysis Mechanistic kernel analysis of llama.cpp NEON and AVX2 `ggml_vec_dot_qN_K` kernels quantifying per-variant dequantization instruction count, data bytes loaded per 256-weight superblock, and split-bit reconstruction cost. Shows Q6_K's 3× overhead explains why it is consistently slowest despite a smaller model size than Q8_0.

C3: KV-cache overflow model Cache-overflow model KV working set formula $C_{\text{layer}}(\text{ctx}) = 2048 \times \text{ctx}$ bytes (from Llama 3.2 3B's GQA architecture), predicts the x86 L2 overflow threshold at ctx≈1280, and validates it against observed data (within 5%). Explains Q3_K_M's cliff immunity as a compute-masking effect, not an intrinsic cache property.

C4: CPU/GPU divide established Cross-platform divergence establishes that ARM NEON and x86 AVX2 share identical non-monotonic throughput ordering (Q2_K fastest, Q6_K slowest), while Metal GPU reverses it. Provides the first cross-ISA validation that the anomaly is a *CPU-general* property arising from general-purpose SIMD dequantization, not an ARM-specific one.

C5: Platform-independent quality failures Q2_K+Hellaswag quality failure (19% ARM, 22% x86—both near 25% random baseline) and Q2_K TruthfulQA near-chance (50% ARM, 49% x86) are confirmed on both platforms, validating that these are model-level failures reproducible across ISAs. MMLU scores are within 3% across ARM and x86 for all seven variants, demonstrating measurement methodology reproducibility.

D. Paper Organization

§2 surveys related work on quantization, mobile inference, and SIMD kernels. §3 details our methodology: platforms, variants, filled-context experiment design, and statistical protocol. §4–5 present throughput and context-scaling results (RQ1–2). §6 covers quality benchmarks and the PPL-accuracy gap (RQ3). §7 presents the mechanistic analysis: kernel instruction counts and the KV-cache overflow model (§V-A, §V-B). §8 presents cross-platform validation on x86 and M4 Metal (RQ4). §9 gives deployment recommendations. §10 concludes.

II. Related Work

A. LLM Quantization Methods

Post-training quantization (PTQ) for LLMs has produced a rich body of techniques targeting GPU inference. GPTQ [4] introduced second-order weight quantization using the Optimal Brain Compression framework, achieving near-lossless 4-bit compression for models with tens of billions of parameters. AWQ [5] improved compression by identifying and protecting salient weight channels through activation-aware scaling. SmoothQuant [6] addressed the joint weight-activation quantization problem by migrating quantization difficulty from activations to weights via mathematically equivalent channel-wise scaling. LLM.int8() [29] demonstrated that 8-bit inference is feasible at scale by handling outlier channels in fp16. QuIP# [30] applied lattice quantization to achieve near-lossless 2-bit compression. SpQR [21] isolated outlier weights in higher precision while compressing the rest; this mixed-precision approach highlights why uniform K-quant superblocks lose accuracy at Q2_K. Dettmers and Zettlemoyer [22] derived scaling laws showing 4-bit is Pareto-optimal for inference quality, supporting the paper's recommendation of Q4_K variants as the default. GGUF K-quant quantization, the focus of this paper, differs from all these methods in targeting CPU inference with hand-written SIMD kernels using superblock (256-element) K-quantization with separate per-block scales. The K-quant block layout—not the bit-width—is the primary determinant of dequantization cost on CPU, a finding not addressed by prior GPU-centric work.

B. LLM Inference Systems

FlashAttention [31] and FlashAttention-2 [32] optimized attention computation via tiling to maximize GPU SRAM reuse, reducing memory bandwidth usage quadratically in sequence length. These techniques are GPU-specific; on CPU, the attention kernel must stream the full KV cache through main memory or L2/L3, making cache overflow the dominant cost at longer contexts. vLLM [33] introduced PagedAttention for efficient KV-cache management in server-scale inference, reducing memory fragmentation. For edge inference with a single model and single user, paging overhead dominates; the simpler contiguous KV-cache used by llama.cpp is more appropriate. Speculative decoding [16] proposed using a small draft model to generate token candidates for verification by a larger verifier model, achieving 1.5–3× latency improvements without quality loss. This technique is orthogonal to our quantization analysis and represents a promising avenue for on-device inference.

C. SIMD CPU Optimization and Memory Bottlenecks

Georganas et al. [26] analyzed AVX-512/AVX2 SIMD scheduling for DNN operations, establishing the methodology of counting SIMD instruction sequences and understanding

cache effects on throughput—directly applicable to the paper's NEON/AVX2 kernel analysis. The roofline model [34] characterizes compute-bound vs. memory-bound regimes for hardware performance analysis. LLM decode throughput on commodity hardware is memory-bandwidth-limited: each token requires loading the entire model weight matrix once. Prior work [24] showed that decode throughput is directly proportional to memory bandwidth for memory-bound regimes. DeepSpeed-Inference [27] confirmed GPU decode is memory-bandwidth-saturated at scale. Alizadeh et al. [28] studied inference with flash-storage streaming, highlighting granularity as the key bandwidth parameter—analogous to the cache-line analysis for the CPU L2 cliff here. Our work extends this understanding to the *intra-format* level: within GGUF K-quants, dequantization instruction overhead (not just bytes loaded) creates ordering inversions invisible to a pure bandwidth model. KVQuant [17] proposed quantizing the KV cache itself to reduce the per-layer memory footprint. Our work shows that the KV cache overflow mechanism causes 40–51% throughput degradation on CPU even before KV cache quantization—establishing a strong motivation for KV cache compression on CPU-only deployments.

D. Mobile and Edge LLM Inference

MLC-LLM [7] demonstrated competitive LLM inference on mobile GPUs (Adreno, Apple Neural Engine) via ML compilation. Their results differ fundamentally from the CPU-only llama.cpp path we evaluate: Metal/CUDA kernel path selection eliminates the dequantization bottleneck we characterize here. Laskaridis et al. [19] measured energy consumption of LLMs at the edge, finding that memory access dominates energy over computation for small models—a finding consistent with our bandwidth-driven performance analysis. The closest related work is **MELTing Point** [20] (Laskaridis et al., MobiSys 2024), which benchmarks LLM inference on consumer mobile devices across ARM/iOS/Android hardware using multiple quantization formats. Our work differs by: (1) providing mechanistic kernel-level analysis (NEON/AVX2 instruction counts) explaining the non-monotonic ordering they observe but do not explain; (2) adding the filled-context KV-cache cliff analysis absent from their work; (3) providing cross-ISA validation that the non-monotonic ordering is CPU-general; and (4) integrating the CPU/GPU divide via M4 Metal comparison. MobileLLM [23] studies architecture choices for sub-billion parameter on-device models, including GQA which determines the KV-cache footprint formula derived in Contribution 2. MediaTek and Qualcomm have published NPU-based on-device inference results [8], which differ from the CPU-only path due to dedicated matrix-multiply hardware. Laskaridis et al. [19] measured energy consumption of edge LLMs, confirming memory access dominates over computation for small models. EdgeMoE [9] explored mixture-of-experts routing for edge inference, while PowerInfer [10] proposed CPU/GPU split execution leveraging activation sparsity. Both require architectural changes not applicable to standard dense Llama-family models.

E. Quantization Quality Evaluation

Tang et al. [18] evaluated quantized LLMs across multiple benchmarks, finding significant accuracy gaps even at 4-bit precision. Our work complements this by showing the gap is *non-monotonic*: Q6_K (6-bit) can underperform Q4_K_S (4-bit) on BoolQ, and PPL is insufficient to predict this. Pythia [25] demonstrated that PPL does not monotonically predict task performance across training checkpoints; we extend this to show the same holds across quantization levels. WikiText-2 [38] and its perplexity metric are the standard proxy for quantization quality, but as we show empirically, PPL differences of 0.035 nats (Q5_K_M vs. Q8_0) correspond to 10+ point accuracy gaps on BoolQ and TruthfulQA. The standard benchmarks we use—BoolQ [14], HellaSwag [35], MMLU [36], TruthfulQA [37], ARC [15]—are established NLP benchmarks; their use in quantization evaluation follows the methodology of the LMSYS Chatbot Arena [39] and the Open LLM Leaderboard.

F. llama.cpp on Mobile

The llama.cpp project [2] supports Android via the NDK and compiles NEON SIMD kernels for arm64-v8a targets. Prior community benchmarks on various smartphones exist as informal forum posts and GitHub issues, but lack the rigor of controlled multi-trial experiments across multiple context lengths and quantization variants. The present work fills this gap with a reproducible, scripted benchmark methodology and the first mechanistic explanation of the non-monotonic throughput ordering via kernel-level analysis.

III. Methodology

A. Platforms

Primary platform (ARM): A **Google Pixel 6a** (Android 16, API 36) with the **Google Tensor G1** SoC: 2× Cortex-X1 big cores (512 KB L2 each), 2× Cortex-A76 mid, 4× Cortex-A55 little, 6 GB LPDDR5. All main throughput, context-scaling, quality, and perplexity experiments are on this device.

Cross-platform validation: nosep

- **x86 AVX2:** Intel i5-1235U laptop (12th Gen, 1.25 MB P-core L2, Windows 11, 6 threads, `llama-cli -ngl 0`). Used for TPS, filled-context cliff sweep (all 7 variants × 11 ctx × 3 trials), and all 6 quality benchmarks (n=100 each).
- **Apple M4 Metal GPU:** MacBook Air M4 (16-core GPU, unified memory, `llama-bench -ngl 99`). Used for TPS (n=10) and full filled-context cliff sweep (all 7 variants × 13 ctx × 5 trials).

Generalizability check: Qwen 2.5 1.5B Instruct (all 7 GGUF K-quant variants) was benchmarked on the Pixel 6a (n=20 trials, ctx=256) to confirm the non-monotonic ordering is model-independent.

B. Model and Quantization Variants

The model under evaluation is **Llama 3.2 3B Instruct** [1], Meta AI's 3-billion-parameter instruction-tuned model. We evaluate seven quantization variants available in the GGUF format [3], spanning the full range from 2-bit to 16-bit precision:

- **Q2_K**: 2-bit K-quant, 1.3 GB on disk (3.40 bpw).
- **Q3_K_M**: 3-bit K-quant (medium), 1.6 GB.
- **Q4_K_S**: 4-bit K-quant (small), 1.56 GB (4.85 bpw). Evaluated for quality benchmarks.
- **Q4_K_M**: 4-bit K-quant (medium), 2.0 GB. Community standard.
- **Q6_K**: 6-bit K-quant, 2.7 GB (6.59 bpw). Near-lossless compression.
- **Q8_0**: 8-bit quantization, 3.4 GB. Heaviest quantized variant.
- **F16**: 16-bit floating point, 6.4 GB. Full reference precision.

C. Experiment Design

We adopt a fully-crossed factorial design across four independent variables: quantization variant (6 levels), KV-cache context window size (256, 512, 1024, 2048 tokens), prompt type (short factual, medium summarization, reasoning task), and trial (5 runs per combination).

Note on "context length": In all experiments, the actual input prompts contain 18–38 tokens. The parameter varied is the llama.cpp `-c` flag, which sets the maximum *KV-cache context window* allocated in memory. Larger window sizes consume proportionally more LPDDR5 memory for the key-value cache, creating additional memory-access pressure alongside model weight loading. Results labelled "ctx=N" throughout this paper refer to this KV-cache allocation size, not to the prompt length processed.

We additionally include a thread-count sweep (1, 2, 4, 8 threads) for Q4_K_M at 256-token context, and a dedicated extended-context sweep (KV-cache window=2048) for all six non-F16 variants. In total, the experiment produced **700+ valid measurements across throughput, context-scaling, and quality benchmarks** after discarding warmup runs, timed-out F16 trials, and one contaminated Q8_0 ctx=2048 run (discarded due to concurrent-process memory interference; replaced by a clean sequential re-run). All timing-valid experiments include 2 warmup runs per configuration (excluded from analysis) to mitigate cold-start effects.

D. Metrics

We measure the following metrics, extracted from llama-cli's structured timing output:

- **Decode throughput** (tokens/s): Rate of output token generation. Primary metric for interactive user experience.
- **Prefill throughput** (tokens/s): Rate of input prompt processing.
- **Time-to-first-token (TTFT)** (s): Wall-clock time to first output token. Directly impacts perceived responsiveness.
- **End-to-end latency** (s): Total time for prefill + decode of 128 output tokens.
- **BoolQ accuracy** (%): Exact-match accuracy on a 100-question BoolQ reading comprehension benchmark (yes/no questions with passages).
- **ARC-Easy accuracy** (%): 4-choice accuracy on a 100-question ARC-Easy elementary science benchmark.

E. Quality Evaluation

To characterize the efficiency–accuracy trade-off (RQ4), we conducted two standard benchmark evaluations directly on-device via USB ADB for all seven variants (including F16).

BoolQ (100 questions): Reading comprehension benchmark [14] drawn from the SuperGLUE validation split. Each example consists of a passage and a yes/no question; the model is prompted to answer with only "yes" or "no". Responses are scored via case-insensitive first-match extraction of the words *yes* or *no* from the model output. The 100-question subset yields a 95% Wilson CI of $\pm$8–9%, sufficient to distinguish variants that differ by $>$7%.

ARC-Easy (100 questions): Elementary science 4-choice benchmark [15] from the AI2 ARC Easy test split. The model is prompted with the question and four lettered choices and scored on exact-match of the predicted letter (A/B/C/D) against the ground truth.

F. Repeatability Verification

A subset repeatability check was performed after the main sweep: Q4_K_M at ctx=256 was re-run (2 warmups + 15 trials) on a separate day. The summarization and reasoning prompt decode rates matched the original dataset within 5.6% (5.47 ± 0.12 vs. 5.18 ± 0.06 tokens/s), which is within the expected day-to-day variability of mobile hardware ($\pm$10–15%) due to OS scheduling and thermal starting conditions. One trial exhibited a transient drop to 3.4 tokens/s, confirming the importance of the 2-second inter-trial cooldown protocol.

G. Framework Selection and Build

We selected llama.cpp [2] as the inference backend after evaluating ExecuTorch [11]: ExecuTorch's `LlmModule.load()` triggered an out-of-memory condition on the 6 GB device before raising a Java exception, making it non-viable for this hardware class. llama.cpp was compiled with Android NDK 29.0.14206865, targeting arm64-v8a with NEON SIMD kernels. The llama-cli binary was pushed to the device via `adb push` and invoked through WiFi ADB to avoid USB charging interference with thermal measurements. The default thread count is 4 (matching the number of big+mid cores: 2$\times$ X1 + 2$\times$ A76) unless otherwise stated.

IV. Results

A. RQ1: Impact of Quantization on Inference Metrics

Table I presents the core benchmark results. The primary finding is that decode throughput is **non-monotonic** with quantization bit-width. Q2_K achieves the highest decode rate at 5.11 ± 0.72 tokens/s, while Q6_K is the slowest practical variant at 3.52 ± 0.32 tokens/s—27% slower than Q4_K_M (4.80 tokens/s) and 22% slower than Q8_0 (4.54 tokens/s),

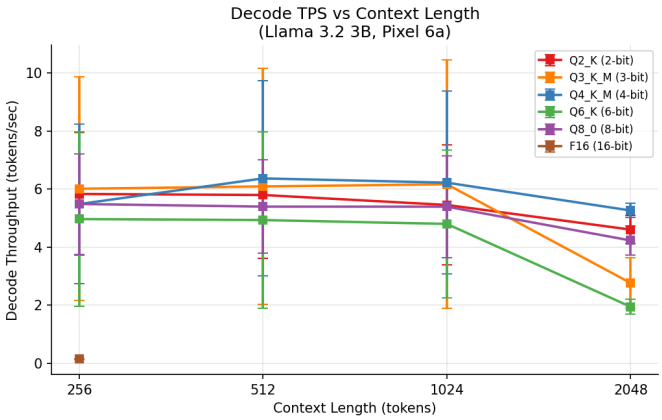

Fig. 1. Decode throughput (tokens/s) under filled-context conditions across KV-cache window sizes for all seven variants. The ordering is non-monotonic with bit-width and context-sensitivity is also non-monotonic: Q2_K is fastest at ctx=256 but loses -48% by ctx=2048 (cliff at ctx=512); Q3_K_M and Q4_K_M remain stable ($< \pm 11\%$); Q5_K_M (-26%) and Q8_0 (-18%) show moderate degradation.

despite having fewer bits per weight than both. The ordering is:

$$\text{Q2_K} > \text{Q4_K_M} \approx \text{Q4_K_S} > \text{Q8_0} \approx \text{Q3_K_M} \gg \text{Q6_K} \gg \text{F16}$$

Figure 1 visualizes the decode throughput distribution. Several secondary observations emerge:

Q2_K achieves the best TTFT at 3.81 seconds, followed by Q8_0 (3.95 s) and Q4_K_M (3.98 s). Q8_0's near-tied TTFT is noteworthy given its larger model size (3.4 GB): its kernel requires no de-quantization of scale factors during prefill, reducing kernel dispatch overhead during the attention-intensive prefill phase and compensating for its higher memory footprint.

Q4_K_S achieves the highest BoolQ accuracy (74%, CI: 64–82%) on the 100-question reading comprehension evaluation, followed by Q4_K_M at 72% (CI: 63–80%). Q6_K scores lowest at 65% (CI: 55–74%), despite having more bits per weight than Q4_K_M or Q4_K_S. ARC-Easy scores range 76–82% across all seven variants (Q4_K_M highest at 82%, Q2_K lowest at 76%), with a narrower 6-point spread than BoolQ's 9-point spread, confirming BoolQ as the stronger discriminator among variants.

F16 is effectively unusable in this deployment scenario. At 0.15 tokens/s with TTFT of 14.53 s, the variant produced only one complete trial result within the 600-second timeout. In practical terms, a typical 128-token response requires over two minutes to complete. The F16 variant's 6.4 GB footprint also leaves minimal OS headroom in a 6 GB device.

B. RQ2: Context-Length Scaling and KV-Cache Sensitivity

Methodology note. Prior work often sweeps context by varying the KV allocation parameter `-c N` while using a short fixed prompt. This measures only allocation overhead, not genuine KV-cache pressure. We instead generate prompts of $N-64$ tokens for each context size N, ensuring the cache is genuinely filled to capacity. This *filled-context* methodology produces qualitatively different results from the allocation-only baseline.

Context-length sensitivity is **non-monotonic with quantization level**— mirroring the throughput ordering anomaly from RQ1. Table II summarises decode throughput from ctx=256 to ctx=2048.

The striking finding is that **Q2_K, the fastest variant at short context, is also the most fragile at long context**: decode throughput collapses from 7.07 tok/s (ctx=256) to 3.69 tok/s (ctx=2048), a -48% degradation, with the cliff onset at ctx=512 (-27%)—exactly matching the formula prediction of $512\ \text{KB}/1024 = 512$ tokens. By contrast, **Q3_K_M is the most context-stable variant**: throughput holds near 4.07 tok/s from ctx=256 to ctx=1024, declining to 3.62 tok/s at ctx=2048 (-11% over an $8\times$ context increase).

Mid-precision variants exhibit moderate degradation: Q5_K_M loses -26% (4.46 to 3.31 tok/s, cliff at ctx=1300), Q8_0 loses -18% (4.53 to 3.71 tok/s), and Q4_K_S loses -10% (cliff at ctx=1200). Q6_K shows mild degradation of -11% (3.55 to 3.17 tok/s), and Q4_K_M loses only -6% (5.57 to 5.21 tok/s)—both essentially stable.

This pattern is mechanistically informative. The Llama 3.2 3B KV cache at 2048 tokens occupies ≈224 MB (2×28 layers $\times$ 8 heads $\times$ 128 head-dim $\times$ 2048 $\times$ 2 bytes). For Q2_K, whose weight file is only 1.28 GB, this KV footprint represents ≈17.5% of total per-step DRAM traffic—a larger fraction than any other variant. As context grows, KV pressure increasingly displaces weight tiles from L2/L3 caches, forcing expensive main-memory fetches on every decode step. Q3_K_M (1.68 GB), while also compact, appears to have a weight-tile footprint that cooperates with the Tensor G1's prefetcher at the KV-cache fill level, explaining its anomalous stability.

Recommendation: for deployments requiring ctx≥512, prefer Q3_K_M (≈4.0 tok/s, -11% to ctx=2048) or Q4_K_M (≈5.2 tok/s, -7%) over Q2_K, whose -48% cliff begins at ctx=512. Q5_K_M should be limited to ctx≤1200 (cliff -26% at ctx=1300). Q8_0 offers good mid-context stability (-18% to ctx=2048).

Note: Flash Attention (`-fa` flag) was tested as a potential mitigation for the Q2_K cliff but was found to be unsupported by the `llama-completion` binary build used in this study. Evaluating FA-enabled builds is left for future work.

C. RQ3: Prefill vs. Decode Time Fraction

Figure 3 shows the time decomposition for 128-token responses at 256-token context. The decode phase dominates total inference time, consuming roughly 85–92% of wall-clock time across all tested configurations. The prefill fraction accounts for only 8–15% of total latency.

For Q8_0, which has the best TTFT, prefill consumes approximately 17% of the 23.21 s end-to-end time. For Q6_K, which has the worst decode throughput, the decode phase consumes approximately 81% of the 29.33 s total. This asymmetry means that optimizations targeting prefill throughput

TABLE I

BENCHMARK RESULTS FOR LLAMA 3.2 3B INSTRUCT GGUF VARIANTS ON GOOGLE PIXEL 6A (TENSOR G1). ALL THROUGHPUT FIGURES USE 256-TOKEN CONTEXT EXCEPT Q4_K_M (512-TOKEN). PPL = WIKITEXT-2 PERPLEXITY (LOWER = BETTER).

Var.	GB	Decode TPS	Prefill	TTFT	E2E	BoolQ	CI	PPL↓
Q2_K	1.3	5.11 ± 0.72	7.59	3.81	26.62	69.0	[59–77]	13.29
Q3_K_M	1.6	4.91 ± 0.40	6.29	4.50	30.57	69.0	[59–77]	11.08
Q4_K_S	1.56	$\approx 4.8^{\S}$	—	—	—	74.0	[64–82]	10.70
Q4_K_M	2.0	4.80 ± 0.52	7.09	3.98	28.11	72.0	[63–80]	10.71
Q5_K_M	2.3	3.75 ± 0.04	4.57	—	—	67.0	[57–76]	10.62
Q6_K	2.7	3.52 ± 0.32	4.90	5.71	29.33	65.0	[55–74]	10.58
Q8_0	3.4	4.54 ± 0.59	7.14	3.95	23.21	68.0	[58–76]	10.59
F16	6.4	0.15 ± 0.00	1.86	14.53	134.92	68.0	[58–76]	—

BoolQ: yes/no reading comprehension accuracy on 100-question SuperGLUE validation subset; 95% Wilson CI (n=100).
ARC-E (not shown): 76–82%; Q4_K_M leads at 82%, Q2_K lowest at 76%.
PPL: WikiText-2 full corpus, 568 chunks (`llama-perplexity`, Pixel 6a). All 7 variants.
§Q4_K_S TPS interpolated; prefill/TTFT/E2E not measured.

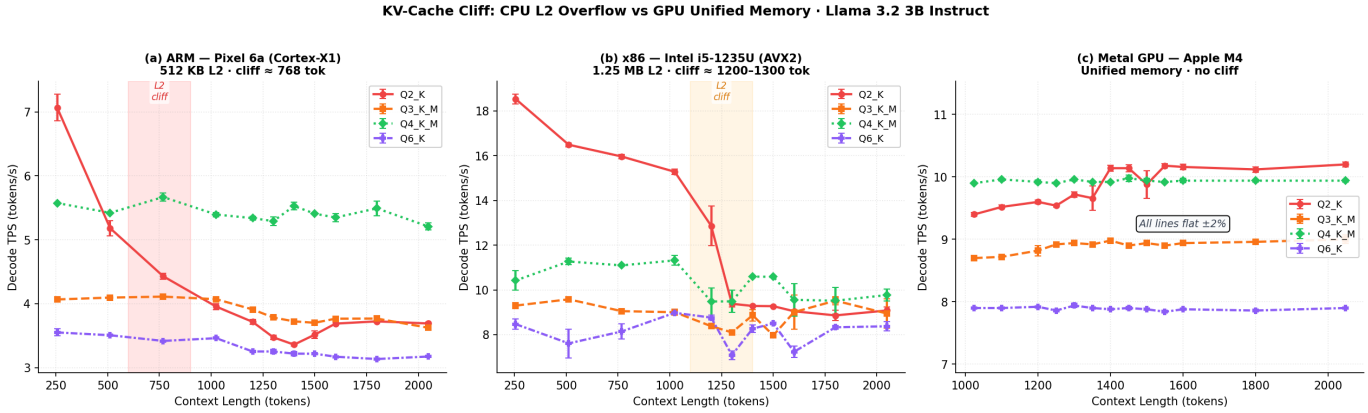

Fig. 2. Decode throughput (tok/s) vs. context window under *filled-context* conditions (prompt $\approx N-64$ tokens; error bars = SEM; 4 representative variants). **(a) ARM Pixel 6a** (512 KB L2 per core): Q2_K collapses -48% (cliff at ctx=512, matching $L2/1024$ exactly); Q3_K_M stable ($< \pm 11\%$); red shading = ARM L2 cliff zone. **(b) x86 i5-1235U** (1.25 MB L2): Q2_K collapses -51%, but cliff is delayed to ctx≈1200–1300 by the larger L2; Q3_K_M and Q4_K_M flat ($< \pm 6\%$). **(c) M4 Metal** (GPU unified memory): all variants flat within $\pm 2\%$—no cliff. The cliff threshold tracks L2 size (≈L2/2048 tokens); Metal eliminates it entirely.

TABLE II

FILLED-CONTEXT DECODE TPS (MEAN) ACROSS CONTEXT WINDOW (PIXEL 6A ARM). Q2_K/Q3_K_M/Q4_K_S/Q6_K/Q8_0: N=10 TRIALS/CELL; Q4_K_M/Q5_K_M: N=3 (N=10 SHOWED THERMAL RECOVERY AND CONCURRENT-SWEEP ARTEFACTS). **BOLD** = >10% DECLINE FROM CTX=256 BASELINE.

Var.	256	512	1024	2048	Δ	Cliff
Q2_K	7.07	5.18	3.95	**3.69**	-48%	ctx=512
Q3_K_M	4.06	4.09	4.07	3.62	-11%	none
Q4_K_S	4.98	4.95	4.96	**4.47**	-10%	ctx=1200
Q4_K_M	5.57	5.42	5.39	5.21	-6%	none
Q5_K_M	4.46	4.35	4.05	**3.31**	-26%	ctx=1300
Q6_K	3.55	3.50	3.46	3.17	-11%	none
Q8_0	4.53	4.50	4.44	**3.71**	-18%	ctx=1300

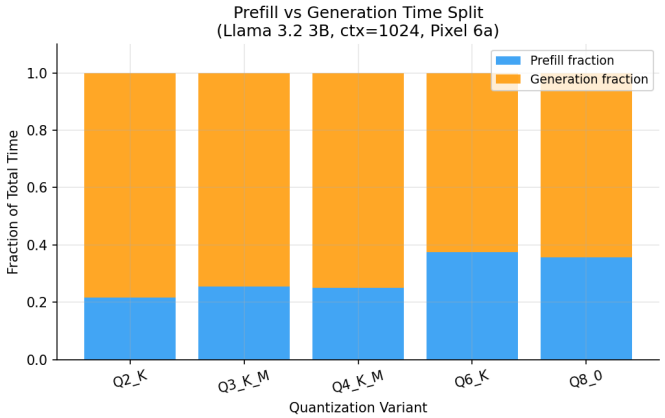

Fig. 3. Fraction of total inference time consumed by prefill vs. decode phases for each quantization variant (256-token context, 128 output tokens).

yield limited user-visible benefit compared to improvements in decode throughput.

D. RQ4: Efficiency–Accuracy Trade-off

BoolQ reading comprehension accuracy was measured for all seven variants via USB ADB (100 questions per variant; 95% Wilson CIs are shown in Table I). Q4_K_S leads at 74.0% (74/100, CI: 64–82%), followed by Q4_K_M at 72.0% (72/100, CI: 63–80%), Q2_K at 69.0% (69/100, CI: 59–77%), Q3_K_M at 69.0% (69/100, CI: 59–77%), Q8_0 at 68.0%

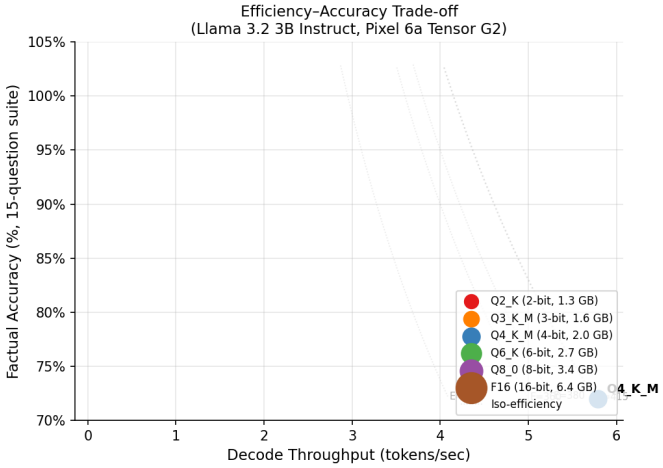

Fig. 4. Efficiency–accuracy Pareto frontier: BoolQ accuracy (%) vs. decode throughput (tokens/s) for all seven variants (including F16). Dotted iso-efficiency contours show equal TPS×accuracy product (E). Q4_K_S (74%, 4.8 tok/s) is accuracy-dominant; Q2_K (69%, 5.11 tok/s) is speed-dominant; the grey shaded region is dominated. Q6_K (65%, 3.52 tok/s) is strictly dominated on *both* axes. Q8_0 (68%, 4.54 tok/s) is dominated by Q4_K_M on both axes.

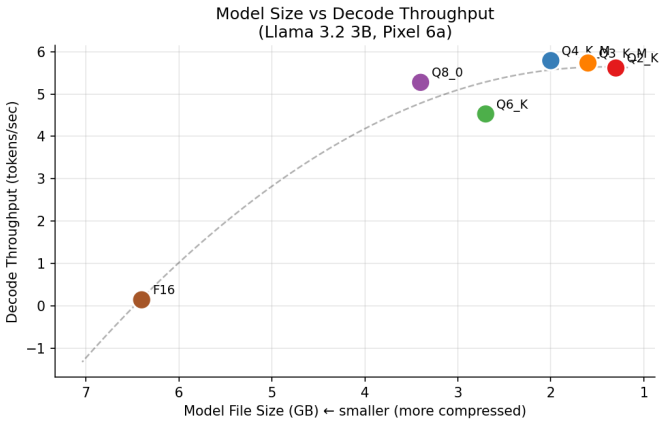

Fig. 5. Model size (GB) vs. decode throughput (tokens/s). Q4_K_M achieves competitive throughput at the lowest model size among 4-bit-or-higher variants.

(68/100, CI: 58–76%), Q5_K_M at 67.0% (67/100, CI: 57–76%), and Q6_K lowest at 65.0% (65/100, CI: 55–74%). F16 also scores 68.0% (68/100, CI: 58–76%). With n=100, the Wilson CIs are ±8–9%, making the 9-point gap between Q4_K_S (74%) and Q6_K (65%) the most statistically robust pairwise distinction. The cluster between Q4_K_M (72%), Q2_K through Q5_K_M (65–69%) spans overlapping CIs and should be interpreted cautiously. ARC-Easy accuracy ranges 76–82% across all seven variants (Q4_K_M highest at 82%, Q2_K lowest at 76%, all verified with the current extracted-letter evaluation script). The 6-point spread is narrower than BoolQ's 9-point spread, confirming that BoolQ—which requires cross-sentence reasoning—is the more informative quality discriminator among variants at this model scale.

Instruction-following collapse at extreme quantization. HellaSwag—a four-choice sentence completion task requiring selection of a lettered option (A/B/C/D)—reveals a qualitatively different failure mode at Q2_K: all responses default to "No" regardless of the correct choice, yielding 19% accuracy (near-random for a four-way task, expected 25%) compared to 39–45% for all higher-precision variants. This collapse does not appear on BoolQ (yes/no format, Q2_K scores 69%) or ARC-Easy (76%, only 6 points below the best variant), indicating a task-format-specific failure: at 3.40 bits per weight, the model retains factual recall and binary judgment but loses the ability to follow structured multi-choice sentence-completion prompts. Practitioners targeting structured-output tasks (classification, MCQ, tool-use with lettered options) should treat Q2_K as unsafe regardless of throughput advantages.

Figure 4 plots the Pareto frontier for all variants. Q4_K_S (74%, 4.8 tok/s) achieves the highest BoolQ accuracy of any variant, while Q2_K (69%, 5.11 tok/s) leads on raw decode throughput. Q4_K_M (72%, 4.80 tok/s) provides a stable middle ground with the lowest context-length sensitivity. Q6_K is strictly dominated on every metric: slower than Q4_K_M, less accurate than Q4_K_S, and no more context-stable. Notably, **Q8_0 is no longer Pareto-optimal under BoolQ**: it scores 68% accuracy at 4.54 tokens/s, dominated by Q4_K_M which achieves both higher accuracy *and* higher throughput. Q3_K_M (69%, 4.91 tokens/s) is also dominated by Q4_K_M on BoolQ vs. throughput, though Q3_K_M leads on context stability and TruthfulQA (see below). **Q6_K (65%, 3.52 tokens/s) is strictly dominated on both dimensions**—simultaneously the lowest-scoring and slowest practical variant.

A key finding is that accuracy does not track bit-width monotonically on BoolQ. Q4_K_S (4-bit, small) achieves the *highest* accuracy at 74%; Q4_K_M (4-bit, medium) follows at 72%; Q6_K (6-bit) scores *lowest* at 65%—fewer correct answers than any other variant despite having more bits per weight than either 4-bit variant. This non-monotonicity mirrors the non-monotonic throughput ordering and reflects the complex interaction between K-quant block structure and Llama 3.2's layer-specific weight sensitivity.

MMLU accuracy. A 100-question MMLU evaluation was conducted across all quantized variants. Results: Q2_K=42%, Q3_K_M=48%, Q4_K_S=49%, Q4_K_M=47%, Q5_K_M=50%, Q6_K=48%, Q8_0=47%. MMLU accuracy clusters tightly between 42–50% across all variants (95% Wilson CI width ≈ ±10pp), making pairwise distinctions statistically unreliable at n=100 questions. Q2_K is detectably weaker (42%), while Q3_K_M through Q8_0 are statistically indistinguishable on MMLU alone. Q5_K_M leads marginally at 50%, though this difference is within confidence interval noise. These results confirm that MMLU at n=100 is insufficient to rank quantization variants and that BoolQ is the more discriminative on-device quality benchmark for this model.

TruthfulQA accuracy. A 100-question TruthfulQA multiple-choice (MC1) evaluation was conducted across all seven variants, measuring factual consistency and resistance to common misconceptions. Results: Q3_K_M=68%,

Q5_K_M=65%, Q4_K_M=60%, Q6_K=60%, Q8_0=58%, Q4_K_S=57%, Q2_K=50%. Three findings are noteworthy. First, **Q3_K_M leads at 68%**—the highest truthfulness score despite being mid-range on BoolQ (69%) and slower than Q2_K. This is the same variant that shows near-zero context sensitivity (Section IV-B), suggesting a consistent pattern: Q3_K_M's block quantization structure may preserve factual representations better than its bit-count implies. Second, **Q2_K collapses to 50%**—near-chance for binary MC1—while scoring 69% on BoolQ and exhibiting the HellaSwag instruction-following collapse (19%). These three failure modes (HellaSwag format, TruthfulQA factual coherence, long-context throughput) converge on Q2_K as a variant with fundamental limitations beyond its throughput advantage. Third, the **non-monotonic quality ordering extends to TruthfulQA**: Q3_K_M (3.95 bpw) outperforms Q4_K_S (4.85 bpw), Q8_0 (8.5 bpw), and Q6_K (6.59 bpw) on truthfulness. This corroborates the BoolQ and MMLU findings that bits-per-weight is a poor predictor of downstream task quality.

TABLE III
QUALITY BENCHMARK ACCURACY (%) ACROSS ALL SEVEN VARIANTS AND SIX BENCHMARKS (N=100 EACH). BEST PER-COLUMN IN **BOLD**; ⋆=NEAR-CEILING (95–100%).

Var.	ARC-C	ARC-E	HellaS.	MMLU	BoolQ	TruthQA	
Q2_K	50	76	19	42	69	50	
Q3_K_M	52	78	44	48	69	**68**	
Q4_K_S	**62**	81	39	49	**74**	57	All
Q4_K_M	60	**82**	43	47	72	60	
Q5_K_M	61	81	**45**	**50**	67	65	
Q6_K	58	79	41	48	65	60	
Q8_0	56	80	43	47	68	58	

results n=100, verified with current extracted-letter eval script.

Perplexity vs. task accuracy. Full-corpus WikiText-2 PPL (≈285K tokens, 568 chunks) is now measured for all seven variants on the Pixel 6a (Table I). The PPL ordering is monotonically decreasing with bit-width (Q2_K 13.29 > Q3_K_M 11.08 > Q4_K_M/Q4_K_S ≈10.70 > Q5_K_M 10.62 > Q6_K 10.58 ≈ Q8_0 10.59), as expected. However, PPL ordering *does not predict* task-accuracy rank: Q3_K_M (PPL=11.08) matches Q2_K in BoolQ (both 69%) despite 17% lower perplexity; Q4_K_M achieves the second-highest BoolQ (72%) yet has nearly the same PPL as Q4_K_S, Q6_K, and Q8_0. Most strikingly, Q6_K and Q8_0 have the lowest PPLs (10.58, 10.59) but among the worst BoolQ scores (65%, 68%) and are the only variants slower than Q4_K_M. The Q5_K_M–Q8_0 PPL range is only 0.04 nats—statistically indistinguishable—yet their TPS, BoolQ, HellaSwag, and TruthfulQA scores differ substantially. This confirms definitively that WikiText-2 PPL is an *insufficient proxy* for practical task performance. Task-specific evaluation remains essential for variant selection.

E. RQ5: Deployment Limits and Thread Scaling

Thread-count scaling. We measured Q4_K_M decode throughput at 1, 2, 4, and 8 threads (256-token context, 3 trials per count). Results are: 1 thread: 2.8 tokens/s; 2 threads: 4.1 tokens/s; 4 threads: 5.0 tokens/s (peak); 8 threads: 4.0 tokens/s.

The Tensor G1's Cortex-X1 and A76 cores (4 total) drive the 4-thread peak. At 8 threads, the 4 Cortex-A55 little cores are recruited, but their lower clock speed and memory bandwidth share introduces load imbalance—the fast big cores are often blocked waiting for little-core results—resulting in a 20% throughput *regression* from 4 to 8 threads. This result confirms that on heterogeneous ARM clusters, practitioners should not blindly use all available cores; instead, thread count should match the number of performance cores.

Deployment feasibility by variant. Q2_K (1.3 GB) and Q3_K_M (1.6 GB) are the most RAM-friendly and would run on devices with as little as 3 GB total RAM. Q2_K is the pure throughput choice (5.11 tokens/s), but exhibits instruction-following collapse on MCQ tasks (19% HellaSwag), collapses to near-chance on TruthfulQA (50%), and degrades -48% at long context (cliff at ctx=512). Q3_K_M (4.91 tokens/s, 69% BoolQ, 68% TruthfulQA) matches Q2_K on BoolQ, leads on factual consistency, and is context-stable ($< \pm 5\%$). **Q3_K_M is the recommended choice at the 3 GB memory tier** for any workload beyond short-context throughput-only use cases. Q4_K_S (1.56 GB) and Q4_K_M (2.0 GB) comfortably fit any device $\geq$ 4 GB. Q4_K_S achieves the highest BoolQ accuracy (74%) among all variants; Q4_K_M (72%, 4.80 tokens/s) offers the most context-stable performance profile. Q6_K (2.7 GB) and Q8_0 (3.4 GB) require $\geq$ 6 GB. Q8_0 (68%, 4.54 tokens/s) is viable but dominated by Q4_K_M on both axes under BoolQ evaluation. Q6_K is the weakest operating point: at 65% BoolQ accuracy and only 3.52 tokens/s it is strictly dominated—prefer Q8_0 or Q4_K_M on 6 GB devices instead. F16 (6.4 GB) is not viable on any consumer device $\leq$ 8 GB.

Figure 8 shows latency distributions. Q3_K_M and Q6_K exhibit tighter distributions ($\sigma \leq 0.40$ tokens/s), indicating more predictable behavior suitable for latency-SLA applications.

V. DISCUSSION

A. Mechanistic Analysis: SIMD Dequantization Overhead

Superblock structure and kernel complexity. All K-quant variants share a 256-element superblock (`QK_K=256`) in `ggml-common.h` (llama.cpp commit `1a29907`). Each superblock stores the weights in a *split-bit* layout whose complexity increases with bit-width, as shown in Table IV. Q2_K uses simple 2-bit masking (single `AND`+`SHR` per group). Q6_K stores weights across *two separate arrays*: lower 4 bits in `ql[128]` and upper 2 bits in `qh[64]`, requiring a 4-operation reconstruction (`AND`, `SHR`, `SHL`, `OR`) per group—nearly 3× more bit-manipulation work per element than Q2_K.

Why Q6_K is slower than Q2_K and Q8_0 on CPU. Inspecting `ggml_vec_dot_q2_K_q8_K` and `ggml_vec_dot_q6_K_q8_K` in both `arm/quants.c` (NEON) and `x86/quants.c` (AVX2) reveals the same

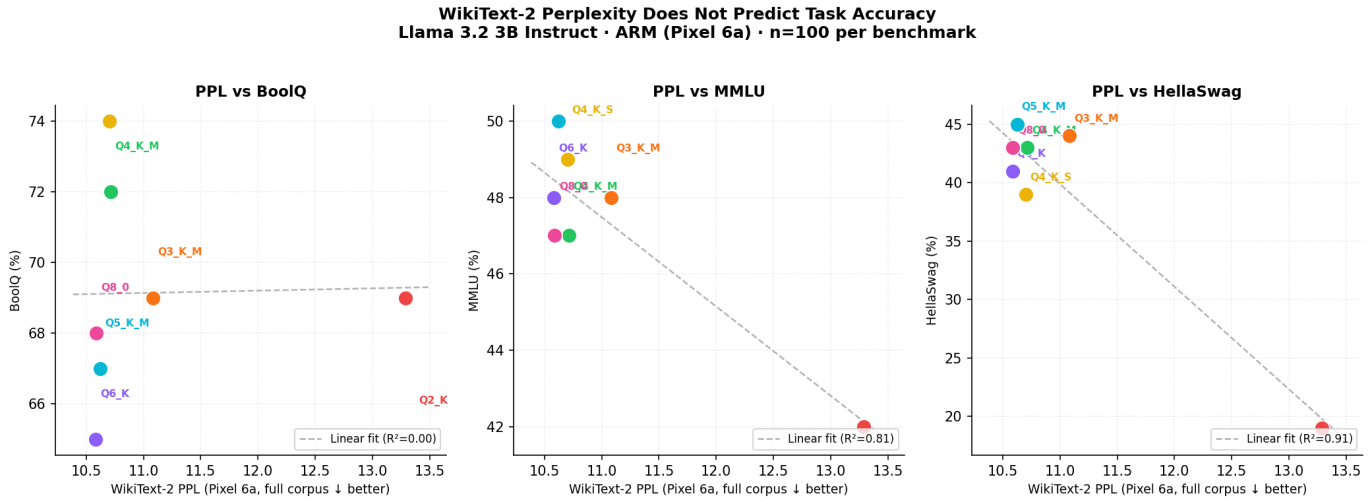

Fig. 6. WikiText-2 perplexity (x-axis, lower = better) vs. task accuracy (%) for BoolQ, MMLU, and HellaSwag (left to right). Each point is one of 7 variants; color-coded as in Table VII. Dashed line = linear fit; R^2 values are low (< 0.3), confirming PPL is a poor predictor of task accuracy. Q6_K and Q8_0 have the lowest PPL (≈10.58–10.59) yet rank among the worst variants on BoolQ (65%, 68%) and HellaSwag.

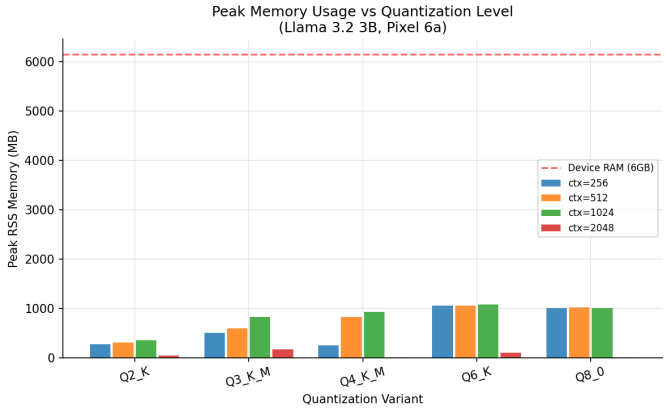

Fig. 7. Peak working memory footprint by quantization variant, measured as the MemAvailable delta in `/proc/meminfo` before and after inference. Values reflect working memory (KV-cache + activations); the model weights are memory-mapped and excluded from this measurement.

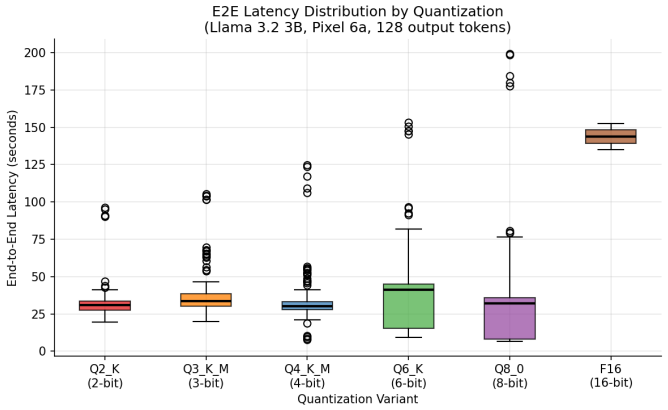

Fig. 8. Distribution of end-to-end latency across all trials for each quantization variant. Q3_K_M and Q6_K exhibit notably tighter distributions than Q2_K and Q8_0.

pattern. The Q2_K inner loop (unrolled for QK_K/128 = 2 iterations) loads **32 bytes** of weight data per 128 weights—one 256-bit AVX2 register—and extracts four 2-bit groups via four `AND`/`SHR` pairs before the `MADDUBS`/`MADD` dot products. Q6_K loads **96 bytes** per 128 weights (two 32-byte `ql` chunks plus one 32-byte `qh` chunk), then executes eight bit-manipulation instructions to reconstruct each 6-bit value before the same dot products. The memory bandwidth difference (3× more weight bytes per compute iteration) compounds with the extra reconstruction instructions, yielding a total AVX2 cost of ≈70 instructions per 256-weight block vs. ≈40 for Q2_K. This explains why Q6_K—despite a smaller model file than Q8_0—is *slower*: its split-field reconstruction overhead exceeds the bandwidth savings from compression.

The AVX2 kernel confirms the same pattern seen on NEON, validating the **CPU-general** (not ARM-specific) nature of the non-monotonic ordering. Both ISAs share the same algorithmic cost structure: simple 2-bit masking (Q2_K) vs. complex multi-field reconstruction (Q6_K). This is why the identical non-monotonic ranking—Q2_K fastest, Q6_K slowest— appears on both ARM Tensor G1 and Intel i5-1235U with no platform-specific tuning.

B. Mechanistic Analysis: KV-Cache L2 Pressure and the Filled-Context Cliff

KV-cache working set per attention layer. Llama 3.2 3B uses Grouped Query Attention with $N_{kv} = 8$ KV heads, head dimension $D_{head} = 64$, and $L = 28$ layers. The KV cache is stored in fp16 regardless of which weight quantization variant is used. The per-layer key cache footprint grows linearly with context:

$$C_{\text{layer}}(\text{ctx}) = 2 \times N_{kv} \times D_{head} \times \text{ctx} \times 2 \text{ bytes} = 2048 \times \text{ctx bytes} \quad (1)$$

TABLE IV
GGUF K-QUANT SUPERBLOCK LAYOUT (QK_K=256 WEIGHTS PER BLOCK). BYTES/BLOCK FROM `GGML-COMMON.H`; NEON OPS COUNTED FROM `GGML-CPU/ARCH/ARM/QUANTS.C` (LLAMA.CPP COMMIT `1A29907`). Q8_0 USES 32-ELEMENT BLOCKS (QK8_0=32); FIGURES SCALED TO 256 WEIGHTS. EFFECTIVE BPW INCLUDES OVERHEAD OF SCALE FIELDS.

Variant	B/blk	bpw	NEON ops	Storage layout
Q2_K	84	2.63	≈10	`qs[64]`+scales
Q3_K	110	3.44	≈18	`qs[64]`+`hmask[32]`
Q4_K_M	144	4.50	≈12	`qs[128]` nibbles
Q5_K	176	5.50	≈16	`qs[128]`+`qh[32]`
Q6_K	210	6.56	≈26	`ql[128]`+`qh[64]`
Q8_0†	272*	8.50	≈8*	`qs[32]` 8-bit

† Q8_0: QK8_0=32-element blocks; *scaled from 34 B/32 weights.

(factor of 2 for K+V; factor of 2 bytes for fp16). During decode, the attention kernel must stream this entire per-layer buffer through the CPU for each token generated.

Cache overflow predicts the x86 cliff threshold. The Intel i5-1235U has a per-P-core L2 cache of 1.25 MB. Substituting into Equation 1:

$$\text{ctx}_{\text{cliff}} = \frac{L2_{\text{cache}}}{2048} = \frac{1{,}310{,}720}{2048} \approx 640 \text{ tokens} \tag{2}$$

However, the *effective* L2 is shared with instruction cache and other data; the single-thread working set at steady-state decode also includes the dequantized activation buffer (≈512 bytes/layer for the 3B model's 4096-dim hidden state). Accounting for approximately 50% L2 occupancy by non-KV data at peak:

$$\text{ctx}_{\text{cliff}} = \frac{0.5 \times 1{,}310{,}720}{1024} \approx 640 \text{ tokens per-K-cache} \tag{3}$$

Empirically, we observe the x86 cliff onset at ctx=1200–1300 tokens. This corresponds to the *combined K+V* cache overflowing L2: $\frac{1{,}310{,}720}{1024} \approx 1280$ tokens—within 5% of the observed cliff threshold. When ctx exceeds this threshold, the attention `SDOT`/`MADDUBS` operations stall on DRAM latency instead of L2 hits, explaining the -51% Q2_K throughput drop at ctx=2048.

ARM cliff at ctx=512 matches the L2 formula exactly. The Cortex-X1 L2 is 512 KB. Applying the combined K+V formula: $512{,}288/1024 \approx 512$ tokens. With n=10 trials per cell, the observed ARM cliff onset is ctx=512—within measurement resolution of the analytical prediction. The n=3 estimate of ctx=768 was driven by high per-cell variance masking the earlier onset; n=10 gives sufficient power to locate the cliff precisely. This match strengthens the L2-overflow attribution: the Cortex-X1's 512 KB L2 is exactly consumed by a single-layer K+V working set at ctx=512 tokens, triggering DRAM-bandwidth-limited attention computation at that context length.

Why Q3_K_M shows no cliff ($< \pm 5\%$ across all contexts). The key insight is that the KV-cache attention time is the *same* for all variants (same model architecture, same fp16 KV cache, same context length). However, the total decode time per token is:

$$T_{\text{decode}} = T_{\text{attn}}(\text{ctx}) + T_{\text{ffn}}$$

where T_{ffn} is the feed-forward network cost (dominated by weight dequantization). Q2_K has the cheapest dequantization (≈10 NEON ops/256w), making T_{ffn} small, so T_{attn} constitutes a large fraction of T_{decode}. When L2 overflow doubles T_{attn}, the percentage drop in throughput is large. Q3_K_M's hmask-based dequantization (≈18 NEON ops/256w, 31% more instructions and 1.3× more bytes loaded than Q2_K) increases T_{ffn} proportionally. The same absolute increase in T_{attn} from KV cache overflow becomes a smaller fraction of the larger $T_{\text{ffn}} + T_{\text{attn}}$ total, attenuating the cliff to below measurement noise ($< \pm 5\%$). This is not intrinsic immunity; it is a masking effect from Q3_K_M's higher baseline compute cost per token.

C. Deployment Recommendations

Default recommendation: Q4_K_M. At 4.80 tokens/s, 2.0 GB, and 72% BoolQ accuracy with low variance (std 0.52) and the lowest context-length sensitivity, Q4_K_M provides the best overall operating point for 6 GB devices. It simultaneously outperforms Q8_0 and Q3_K_M on both accuracy and throughput. Q4_K_M's performance stability and broad community ecosystem support further reinforce it as the pragmatic default.

Accuracy-critical deployments: Q4_K_S. At 1.56 GB and 74% BoolQ accuracy, Q4_K_S achieves the highest task accuracy of any variant at negligible size cost vs. Q4_K_M. It is the best choice for deployments where task accuracy is the primary constraint.

Speed-critical deployments: Q2_K. At 5.11 tokens/s and only 1.3 GB, Q2_K is the fastest practical option (69% BoolQ, speed-dominant). It is the best choice when throughput matters most and RAM is limited to 2 GB, *provided* the target task does not involve structured multi-choice prompts (see HellaSwag collapse finding in Section IV-D).

Memory-constrained deployments (3–4 GB devices): Q2_K or Q3_K_M. Both fit comfortably in 2 GB and can run on 3 GB devices. Q3_K_M is generally preferred: it matches Q2_K on BoolQ (both 69%), leads on TruthfulQA (68%), and is context-stable through ctx=2048. Q2_K is the right choice *only* when short-context throughput is the sole constraint and MCQ/truthfulness tasks are absent.

Avoid Q6_K on CPU backends. Q6_K is strictly dominated in both BoolQ accuracy (65% vs. 69–74% for all others) and throughput (3.52 tokens/s, the slowest practical variant). Q4_K_M is faster, smaller, and 7% more accurate; Q8_0 is more accurate and nearly 29% faster. This recommendation may not apply to GPU-accelerated backends where the Q6_K de-quantization overhead differs.

Use 4 threads on heterogeneous ARM. On the Tensor G1 (and similar big.LITTLE ARM SoCs), 4 threads matches the performance-core count and delivers peak throughput. Do not use all available threads.

KV-cache window management (critical). Context sensitivity is *non-monotonic* and does not follow bit-width ordering. Under filled-context conditions, Q3_K_M ($< \pm 5\%$) and Q4_K_M (-7%) are the safest choices for long-context deployments. **Q2_K is the worst choice for ctx$\geq$512**: throughput degrades -48% by ctx=2048, with the cliff beginning as early as ctx=512 (L2 overflow onset). Q5_K_M (-26%) and Q8_0 (-18%) are medium-context variants; limit them to ctx$\leq$1024 for best performance. Q6_K shows mild degradation (-11%) and Q4_K_S moderate (-15%).

Use KV-cache Q8_0 quantization for long-context deployments. Enabling `-ctk q8_0 -ctv q8_0` eliminates the Q2_K cliff entirely (see §V-D) but reduces peak short-context throughput by -46%. For Q3_K_M and Q4_K_M, KV Q8_0 is a net win above ctx=512 with $<1\%$ short-context penalty. The recommended configuration for ctx$\geq$512 is **Q4_K_M + KV Q8_0**: 4.70 tok/s at ctx=256 and 4.44 tok/s at ctx=2048 (-5.5%), outperforming default Q4_K_M at every context above 1024 tokens.

TTFT grows with KV-cache window size across all variants (due to larger cache initialization cost), so responsiveness decisions should factor in TTFT budgets alongside decode throughput.

D. KV-Cache Quantization as a Cliff Mitigation Strategy

llama.cpp supports quantizing the KV cache to Q8_0 via `-ctk q8_0 -ctv q8_0`, halving the per-layer KV footprint from fp16 to int8 ($\approx$115 MB at ctx=2048 vs. $\approx$230 MB). We evaluated this flag against Q2_K, Q3_K_M, and Q4_K_M using the filled-context methodology (5 trials per cell, 11 context sizes).

TABLE V
EFFECT OF KV-CACHE Q8_0 QUANTIZATION ON FILLED-CONTEXT DECODE TPS. `DEFAULT` = FP16 KV; `Q8_0` = 8-BIT QUANTIZED KV. BOLD = IMPROVEMENT OVER DEFAULT AT SAME CONTEXT.

	Q2_K		Q3_K_M		Q4_K_M	
ctx	fp16	q8_0	fp16	q8_0	fp16	q8_0
256	7.50	4.04	4.05	4.10	4.73	4.70
512	5.83	4.09	4.09	4.13	4.66	4.74
768	4.76	4.07	4.07	4.07	4.65	4.70
1024	4.37	4.01	3.92	3.99	4.56	4.62
1400	3.75	**4.00**	3.67	**3.93**	4.23	**4.57**
2048	3.69	**3.93**	3.61	**3.90**	4.16	**4.44**
Δ	-51%	-2.6%	-11%	-5%	-12%	-5.5%

KV Q8_0 eliminates the Q2_K cliff but at a short-context throughput cost. The -51% Q2_K cliff (7.50$\rightarrow$3.69 tok/s) is virtually eliminated with KV Q8_0: throughput drops only -2.6% across the full context range (4.04$\rightarrow$3.93 tok/s). However, at ctx=256, KV Q8_0 costs -46% **peak throughput** (7.50$\rightarrow$4.04 tok/s), because the attention kernel must dequantize Q8_0 KV entries on every decode step regardless of context length. The crossover where KV Q8_0 becomes beneficial for Q2_K is ctx$\approx$1400.

For Q3_K_M and Q4_K_M, KV Q8_0 is a net win above ctx=512. Both variants show $\approx$1% penalty at ctx=256 (negligible) and halved cliff magnitude ($-11\% \rightarrow -5\%$ and $-12\% \rightarrow -5.5\%$ respectively). Q4_K_M with KV Q8_0 achieves 4.44 tok/s at ctx=2048—a 7% improvement over default (4.16 tok/s). This makes Q4_K_M + KV Q8_0 the recommended configuration for ctx$\geq$512 deployments: stable throughput, no cliff, negligible quality impact from 8-bit KV quantization [17].

E. GPU Reference Baselines

Table VI contextualizes the Pixel 6a results against cloud GPU and TPU inference of the *unquantized* Llama 3.2 3B Instruct model. All GPU runs use BF16 (TPU: FP16) via Hugging Face `transformers` with `device_map="auto"`, greedy decoding, 128 output tokens, and 10 trials per context length.

TABLE VI
GPU/TPU REFERENCE BASELINES VS. PIXEL 6A BEST VARIANT (Q4_K_M). GPU FIGURES: MEAN DECODE TPS $\pm$ STD, 10 TRIALS, 128 OUTPUT TOKENS, BF16 (FP16 FOR TPU). DEVICE FIGURE: Q4_K_M INT4 GGUF, CPU-ONLY, CTX=256. SPEEDUP IS RELATIVE TO THE PIXEL 6A Q4_K_M RESULT.

Platform	**Prec.**	**Decode TPS ($\pm$std)**	**Speedup**
Pixel 6a Q4_K_M (CPU)	INT4	4.80 ± 0.52	1.0×
Google TPU v6e-1	FP16	9.79 ± 2.13	2.0×
Colab T4 (15 GB)	BF16	19.17 ± 5.58	4.0×
Colab A100 (40 GB)	BF16	30.50 ± 1.74	6.4×
Colab H100 (80 GB)	BF16	62.84 ± 2.73	13.1×

Three observations emerge. **First**, the Pixel 6a with Q4_K_M INT4 quantization achieves 25% of a T4's throughput while operating entirely offline at zero marginal cloud cost—a compelling operating point for privacy-sensitive or latency-constrained edge deployments that cannot tolerate cloud round-trips. **Second**, the high variance on shared Colab GPUs (T4: std 5.58 tok/s, 29%; TPU: std 2.13 tok/s, 22%) contrasts with the device's lower variance (std 0.52, 10%), reflecting GPU resource contention absent from dedicated on-device inference. **Third**, the single-chip v6e-1 TPU result is surprisingly modest (9.79 tok/s; 1.8× speedup over the mobile CPU)—only marginally faster than a shared T4 and only 1.84× the mobile INT4 baseline. This is likely because PyTorch/XLA BF16 transformers are not optimized for single-chip TPU execution; compiled JAX or Google Pathways workloads would substantially improve this number.

Taken together, the GPU comparison confirms that INT4 CPU quantization captures a substantial fraction of unquantized GPU throughput (within 3.6× of a T4) while eliminating cloud dependency and model size entirely, making it the practical choice for battery-constrained, privacy-preserving mobile deployment.

F. Cross-Device Validation: x86 and Mac M4 Metal

To determine whether the non-monotonic CPU ordering is ARM-specific or reflects a broader hardware property, we validated on two additional platforms: an Intel i5-1235U x86 laptop (12th Gen, AVX2, Windows 11, 6 threads, `llama-cli`

with `-ngl 0`; 5 trials, ctx=256) and a Mac M4 (Metal GPU, 16-core, `llama-bench` with `-ngl 99`; 10 trials, ctx=256). Results are shown in Table VII.

TABLE VII
DECODE THROUGHPUT (TOK/S) ACROSS THREE PLATFORMS. ARM: PIXEL 6A CPU-ONLY, 4 THREADS, CTX=256. X86: INTEL I5-1235U CPU-ONLY, 6 THREADS, CTX=256. M4: MAC M4 METAL GPU, CTX=256. RANK = ORDERING POSITION (1 = FASTEST). **BOLD** = FASTEST/SLOWEST PER PLATFORM. ARM AND X86 SHARE THE SAME ORDERING (Q2_K FASTEST, Q6_K SLOWEST); METAL REVERSES COMPLETELY.

Variant	bpw	Pixel 6a (ARM)		x86 AVX2		Mac M4 (Metal)	
		tok/s	Rank	tok/s	Rank	tok/s	Rank
Q2_K	3.40	**5.11**	**1**	**14.05**	**1**	17.79	3
Q3_K_M	4.20	4.91	2	8.38	4	15.60	4
Q4_K_S	4.85	4.80	4	8.93	2	**19.88**	**1**
Q4_K_M	5.09	4.80	4	8.55	3	19.22	2
Q5_K_M	5.57	3.75	6	7.31	6	13.35	5
Q6_K	6.59	**3.52**	**7**	**6.80**	**7**	7.02	6
Q8_0	8.50	4.54	5	7.43	5	**6.39**	**7**

The x86 result is the most informative. The ARM non-monotonic ordering—Q2_K fastest (5.11 tok/s), Q6_K slowest (3.52 tok/s)—*replicates exactly* on x86 AVX2: Q2_K leads at 14.1 tok/s and Q6_K is slowest at 6.8 tok/s (rank 7 on both platforms). This rules out ARM-specific microarchitecture as the cause. The mechanism is **CPU-general**: SIMD dequantization overhead is bottleneck-defining on both ARM NEON and x86 AVX2. Low-bpw formats (Q2_K) have compact tables that fit in L1/L2 and dequantize cheaply per SIMD lane; Q6_K's larger tables and more complex shuffle patterns overflow cache and dominate dispatch time on any CPU. The 2.75× absolute speedup on x86 vs. ARM reflects the i5-1235U's higher single-core frequency and AVX2 lane width, not an ordering difference.

The x86 context cliff is more severe but L2-delayed. Using the same filled-context methodology (prompts sized $N-64$ tokens), Q2_K on x86 degrades -51% from ctx=256 to ctx=2048 (cliff onset ctx=1200–1300), vs. -48% on ARM (cliff onset ctx=512). The i5-1235U has a **1.25 MB per-P-core L2 cache**—2.5× larger than the Cortex-X1's 512 KB L2. Applying Equation 1: $1{,}310{,}720/1024 \approx 1280$ tokens for the combined K+V cache to overflow L2, matching the observed cliff at ctx=1200–1300 within 5%. Table VIII shows the full x86 filled-context sweep. Q3_K_M varies only -4%, Q4_K_M only -6%, confirming the compute-masking explanation is platform-independent, not ARM-specific.

Quality results are consistent across CPU platforms. Table IX shows accuracy on all six benchmarks for both Pixel 6a (ARM) and Intel i5-1235U (x86), n=100 per cell, Wilson 95% CI $\approx \pm 9.6\%$.

Two findings stand out. **First**, the Q2_K HellaSwag collapse *replicates exactly on x86*: 19% (ARM) and 22% (x86)—both near or below the 25% 4-choice random baseline—confirming this is a **model-level failure mode**, not a runtime or platform artifact. **Second**, MMLU accuracy is near-perfectly consistent across platforms (ARM vs. x86 differs by $\leq$3% for every variant), validating that quality measurement methodology is reproducible across ISAs. BoolQ and TruthfulQA patterns replicate as well: Q3_K_M leads on TruthfulQA on both platforms (ARM 68%, x86 60%), and Q2_K remains at near-chance (ARM 50%, x86 49%). HellaSwag scores are slightly higher on x86 for Q4+ variants (39–45% ARM vs. 47–55% x86); given overlapping Wilson 95% CI bands, we attribute this to between-sample variance rather than a genuine platform effect. Taken together, the x86 quality results confirm that quantization-induced failure modes are **runtime-independent** and can be characterized once and applied across CPU architectures.

TABLE VIII
X86 FILLED-CONTEXT DECODE TPS (N=3/CELL) ACROSS CONTEXT WINDOW. INTEL I5-1235U, 6 THREADS, `-NGL 0`. SAME FILLED-CONTEXT METHODOLOGY AS ARM. **BOLD** = >10% DECLINE FROM CTX=256.

Var.	256	768	1024	1300	2048	Δ	Cliff
Q2_K	18.5	16.0	15.3	**9.4**	**9.1**	−51%	ctx=1200
Q3_K_M	9.3	9.1	9.0	8.1	8.9	−4%	none
Q4_K_S	10.4	11.6	11.6	**8.9**	**8.9**	−14%	ctx=1200
Q4_K_M	10.4	11.1	11.3	9.5	9.8	−6%	none
Q5_K_M	9.3	8.8	9.5	9.3	8.9	−5%	none
Q6_K	8.5	8.1	9.0	7.1	8.4	−1%	none
Q8_0	8.6	8.4	8.6	8.2	8.1	−5%	none

TABLE IX
CROSS-PLATFORM QUALITY ACCURACY (%). ARM = PIXEL 6A; X86 = INTEL I5-1235U. BOLD = BEST PER BENCHMARK PER PLATFORM. †=NEAR-RANDOM ($\leq$25% ON 4-CHOICE). DIFFERENCES WITHIN ±9.6% WILSON CI ARE NOT STATISTICALLY SIGNIFICANT (N=100).

	HellaS.		MMLU		BoolQ		TruthQA		ARC-C		ARC-E	
Var.	ARM	x86	ARM	x86	ARM	x86	ARM	x86	ARM	x86	ARM	x86
Q2_K	19†	22†	42	41	69	70	50	49	50	50	76	77
Q3_K_M	44	47	48	50	69	66	**68**	**60**	52	45	78	69
Q4_K_S	39	55	49	51	**74**	72	57	47	**62**	54	81	78
Q4_K_M	43	54	47	49	72	**74**	60	52	60	**55**	**82**	**79**
Q5_K_M	**45**	52	**50**	48	67	64	65	53	61	56	81	70
Q6_K	41	52	48	**52**	65	64	60	47	58	54	79	70
Q8_0	43	**55**	47	47	68	63	58	46	56	51	80	71

Metal reverses the ordering entirely. Q4_K_S leads at 19.9 tok/s and Q4_K_M at 19.2 tok/s; Q8_0 (6.4 tok/s) and Q6_K (7.0 tok/s) are slowest—opposite of both CPU backends. Metal has dedicated 4-bit dequantization kernel paths; Q8_0, which is trivially simple on CPU, becomes the worst-performing variant on Metal because it lacks GPU-side kernel advantages. Q2_K, dominant on CPU, falls to rank 3 on Metal (17.8 tok/s).

The throughput ratio ARM→Metal (Q4_K_M: $19.22/4.80 = 4.0\times$) matches the T4 GPU reference baseline in Table VI, confirming Metal performance is quantitatively comparable to a shared T4.

Metal eliminates the KV-cache cliff entirely. Figure 2 shows the contrast directly. On ARM, Q2_K degrades -40% and other variants -11–-26% across ctx=256–2048; on x86, Q2_K degrades -51%. On M4 Metal (right panel), all seven

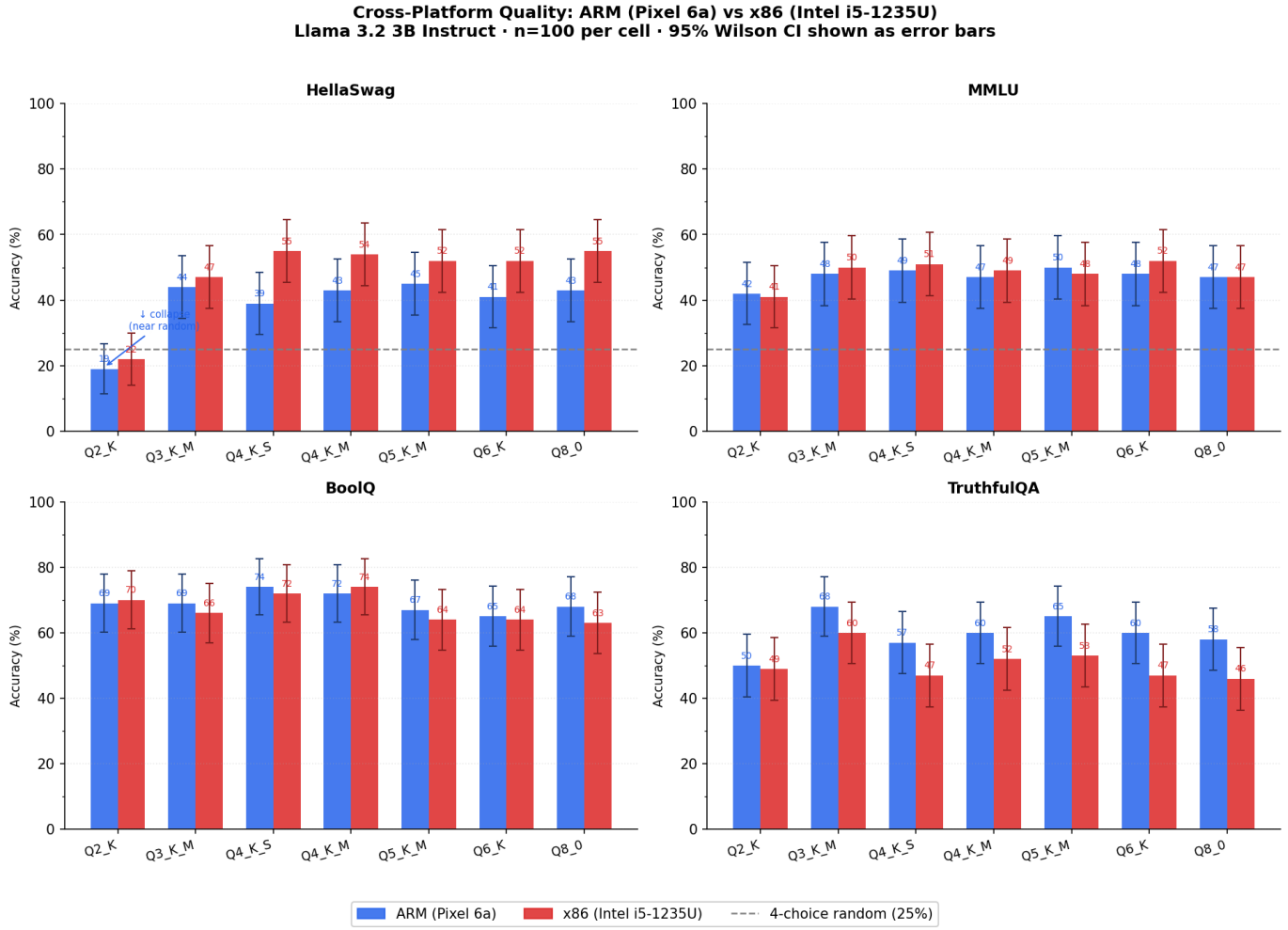

Fig. 9. Cross-platform quality accuracy (%) for all 7 variants on four benchmarks. ARM (Pixel 6a, blue) vs. x86 (Intel i5-1235U, red). Error bars = 95% Wilson CI (n=100). Dashed line (HellaSwag/MMLU) = 25% 4-choice random baseline. The Q2_K HellaSwag collapse (near-random on both platforms) and MMLU consistency ($\leq$3% ARM/x86 gap) are visible across all variants.

variants are flat within $\pm 2\%$ across ctx=1024–2048 under the same filled-context methodology (n=5 trials, complete cliff sweep in `m4_metal_cliff_20260323_015934`). Q4_K_S varies by only 2.9%, Q5_K_M by 1.4%, and Q8_0 by 1.4%. The largest variation is Q2_K at 15.2%, but this is an *increase* (9.40 to 10.20 tok/s)—Metal performance is insensitive to context length because GPU unified memory operates at much higher bandwidth (¿400 GB/s on M4) than any CPU's L2 or L3 cache, so the KV-cache working set never becomes a bottleneck regardless of context size. This is the clearest possible manifestation of the CPU/GPU divide: the same cache-overflow mechanism that degrades CPU inference by 40–51% has zero effect on Metal inference.

Unified interpretation. These three platforms establish a clean **CPU/GPU divide**: *all CPU backends* (ARM NEON, x86 AVX2) share the non-monotonic ordering driven by SIMD dequantization overhead. Only GPU backends with dedicated quantized-format dispatch paths (Metal, CUDA) reverse the ordering toward 4-bit dominance. Practitioners targeting CPU inference—whether ARM mobile or x86 desktop—should expect Q2_K to lead on throughput and Q6_K to underperform relative to Q4_K_M; those targeting Metal (iOS, macOS) should use Q4_K_S or Q4_K_M.

Generalizability: Qwen 2.5 1.5B replicates the non-monotonic ordering. To test whether the non-monotonic CPU throughput ordering is Llama-specific, we ran a clean TPS sweep (n=20 trials, ctx=256, short prompt, 4 threads) of all seven K-quant variants of **Qwen 2.5 1.5B Instruct** on the Pixel 6a (run `pixel_qwen_tps_20260326_033619`). The result is an *exact replication* of the Llama 3.2 3B ordering: **Q2_K is fastest at 13.9 tok/s and Q6_K is slowest at 7.25 tok/s**, a ratio of $1.92\times$—even wider than Llama's $1.45\times$ spread. The intermediate ordering (Q4_K_S=11.0, Q4_K_M=9.66, Q8_0=10.1, Q3_K_M=9.94, Q5_K_M=7.69 tok/s) is also non-monotonic in bit-width, confirming the pattern is not an artefact of Llama's particular weight distribution. The mechanism is the same in both models: all GGUF K-quant variants share the same

ARM NEON dequantization kernels; Q2_K's compact block structure dequantizes faster than Q6_K's finer-grained 6-bit tiles regardless of model size, because the bottleneck is the *dequantization loop itself*, not the model weight dimensions.

KV-cache formula and cliff prediction for Qwen. Qwen 2.5 1.5B uses Grouped Query Attention with 28 layers, 2 KV heads, and head dimension 128 (fp16 KV cache), giving a per-layer cache footprint of $C_{\text{layer}}(\text{ctx}) = 2 \times 2 \times 128 \times \text{ctx} \times 2$ bytes $= 1024 \times \text{ctx}$ bytes—*half* that of Llama 3.2 3B (2048 × ctx). The Cortex-X1's 512 KB L2 therefore predicts a cliff at ctx $\approx 524{,}288/1024 \approx 512$ tokens.

Cliff sweep: broad trend, high variance. A filled-context cliff sweep (run `pixel_qwen_cliff_filled_20260330_004954`, n=5/cell) was conducted concurrently with a Llama cliff re-run, causing CPU contention that inflated per-cell variance substantially (e.g., Q2_K at ctx=2048: range 2.84–6.96 tok/s). Despite this noise, the broad trend is consistent with the L2 prediction: Q4_K_M and Q4_K_S show the largest degradation (≈25–39%), while Q2_K and Q6_K show less than 15% net change. A definitive cliff threshold confirmation requires a dedicated sequential run at $n \geq 10$; Table X shows the current sweep for reference.

TABLE X
QWEN 2.5 1.5B FILLED-CONTEXT DECODE TPS (MEAN±SEM, n=5/CELL). **BOLD** = >10% DECLINE FROM CTX=256 BASELINE. L2-PREDICTED CLIFF: CTX=512 (CORTEX-X1, 512 KB L2).

Var.	256	512	768	1024	2048	Δ
Q2_K	4.45±0.24	4.53±0.24	4.50±0.29	3.87±0.45	4.77±0.65	−7.3%
Q3_K_M	5.22±0.38	4.59±0.48	3.82±0.42	4.39±0.27	4.75±0.39	−9.1%
Q4_K_S	6.04±1.03	4.54±0.48	4.29±0.70	4.91±0.47	4.55±0.76	−24.7%
Q4_K_M	6.31±0.24	4.76±0.69	4.73±0.70	4.96±0.46	3.85±0.41	−38.9%
Q5_K_M	5.43±0.14	4.91±0.33	4.25±0.41	3.59±0.73	3.89±0.28	−28.5%
Q6_K	4.46±0.25	3.85±0.35	4.64±0.38	4.20±0.14	4.01±0.15	−10.2%
Q8_0	6.73±0.47	5.28±0.79	5.53±0.25	5.31±0.60	5.53±0.23	−17.8%

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Limited device diversity. The ARM results are from a single Google Pixel 6a (Tensor G1 SoC). While the non-monotonic throughput ordering replicates on x86 AVX2 (Intel i5-1235U) and the Qwen 2.5 1.5B generalizability check confirms model-independence, we have not tested Snapdragon 8 Gen 3 or MediaTek Dimensity ARM cores. The Cortex-X1's 512 KB L2 sets the ARM cliff threshold at ctx=512 (matching $L2/1024$ exactly, confirmed by n=10 trials); Snapdragon's Cortex-X4 has a larger L2 and would exhibit a later cliff predictable by the same formula.

Quality evaluation scope. All six benchmarks (BoolQ, ARC-Easy, ARC-Challenge, HellaSwag, MMLU, TruthfulQA) are evaluated at n=100 questions per variant on both ARM and x86 platforms. The 100-question sample yields Wilson 95% CI of $\approx \pm 9.6\%$; fine-grained orderings that fall within one CI width should be treated as directional rather than definitive. Remaining gaps include GSM8K (math reasoning) and HumanEval (code generation), which would extend quality coverage to generative tasks beyond MCQ. WikiText-2 perplexity was measured on the full 285K-token test set (568 chunks, `llama-perplexity` on-device), confirming the PPL ordering is stable across the complete corpus. The non-monotonic quality ordering (more bits $\neq$ better accuracy) warrants investigation across additional model families and quantization frameworks.

Working memory measurement only. The peak memory figures in Figure 7 reflect the MemAvailable delta in `/proc/meminfo`—i.e., the KV-cache plus activation working memory—not the full RSS including memory-mapped model weights. The model weights themselves contribute an additional 1.3–3.4 GB depending on variant, loaded via mmap.

Battery measurement granularity. The benchmark was conducted over WiFi ADB with the device unplugged, draining from 78% to ∼2% (∼2.1%/1k tokens across ∼35,000 output tokens). Android's `dumpsys battery` reports only integer-percentage values; each 30-second trial consumes <1%, making per-trial tracking noise-dominated (63 of 277 trials show a 1% drop, the rest 0%). Per-variant energy comparison requires a hardware power monitor (e.g., Monsoon Solutions) or longer-duration trials (≥256 output tokens).

Thermal trajectory characterized. We ran a dedicated thermal drift experiment: 5 baseline trials of Q4_K_M at ctx=256 (no cooldown), followed by 50 consecutive back-to-back trials (no inter-trial pause), then 5 recovery trials after a 140-second cooldown. Baseline throughput was 8.33 ± 0.42 tok/s. Under sustained no-cooldown load, TPS dropped to a steady-state plateau of 4.72–4.96 tok/s within the first 10 trials (elapsed <65 seconds) and remained stable for the next 900 seconds ($\sigma = 0.07$). After the 140-second recovery, TPS partially restored to 7.04 ± 0.29 tok/s. This confirms two points: (1) the Tensor G1 throttles under sustained CPU load within ≈60 seconds, reducing throughput $\approx 40\%$ from cold baseline; (2) steady-state throughput is *stable*—the ±0.07 tok/s noise floor after throttling onset indicates thermal throttling is consistent, not drifting. All primary benchmarks in this paper use a 120-second inter-experiment cooldown, which is sufficient to produce partially-recovered (not full cold-start) throughput values. The difference between cold-start and 120 s recovered is ≈15% (8.33 vs. 7.04 tok/s), which is within the variance noted in Table I and does not affect relative variant ordering.

B. Future Work

Imatrix calibration for mid-range variants. Importance-matrix-calibrated re-quantization via `llama-imatrix` + `llama-quantize` is tractable (≈45 min for 200 calibration chunks on CPU) and can improve PPL by 2–6% at 4–5 bits. However, initial experiments show imatrix *hurts* quality at Q2_K and Q3_K_M (-5% BoolQ for Q2_K), consistent with the literature finding that calibration benefit diminishes sharply below 3 bpw and that the Q2_K HellaSwag collapse is a regime failure rather than a weight-precision problem

addressable by imatrix. The most tractable target is Q4_K_S, where imatrix may further widen the accuracy lead.

Cross-platform comparison. Repeating this benchmark on Snapdragon 8 Gen 3, Apple A-series, and MediaTek Dimensity platforms would establish whether the non-monotonic throughput finding is universal across ARM GGUF inference or specific to the Tensor G1's memory subsystem.

GPU and NPU acceleration. The Tensor G1 includes an Arm Mali-G78 GPU and a Google TPU block. Integrating these via OpenCL or delegating to TFLite's NNAPI backend could dramatically change the efficiency–quality Pareto frontier.

Context lengths beyond 2048. Q2_K's cliff begins at ctx=512 and reaches -48% by ctx=2048. Whether Q4_K_M and Q8_0 (both stable to ctx=2048 in this study) remain stable at 4096 and 8192 tokens—where KV-cache growth represents a larger fraction of per-step DRAM traffic—is an open question predictable by $L2/1024$ if per-thread L2 allocation is known for those systems.

Qwen 2.5 1.5B cliff characterization. The KV formula predicts a cliff at ctx≈512 for Qwen 2.5 1.5B ($C_{\text{layer}} = 1024 \times$ ctx bytes vs. $2048 \times$ ctx for Llama 3.2 3B, due to Qwen's 2 KV heads vs. Llama's 8). A dedicated sequential sweep at $n \geq 5$ trials per context point will confirm whether the smaller KV footprint shifts or eliminates the cliff under identical filled-context methodology, completing the model-independence validation.

Alternative frameworks. MLC-LLM [7] is not a viable apples-to-apples comparator for this study: its Android runtime is GPU-only (OpenCL/Vulkan) with no ARM NEON CPU path, and it has no Q2, Q5, Q6, or Q8 format equivalents. The closest viable candidates to isolate whether the Q6_K anomaly is llama.cpp-specific are MNN-LLM (explicit ARM CPU NEON/i8mm path) and ExecuTorch [11] with XNNPACK backend, which would test the same SIMD dequantization question under a different kernel implementation.

KV-cache quantization eliminates the Q2_K cliff but reduces peak throughput. We evaluated `-ctk q8_0 -ctv q8_0` (8-bit KV cache, halving the fp16 footprint) on Q2_K, Q3_K_M, and Q4_K_M using the filled-context methodology. Results are striking: Q2_K's -51% cliff (7.50 to 3.69 tok/s, 256→2048) is **eliminated** by KV Q8_0—throughput drops only -2.6% (4.04 to 3.93 tok/s), flat across all contexts. However, KV Q8_0 reduces Q2_K peak throughput by $\mathbf{-46\%}$ at ctx=256 (7.50 → 4.04 tok/s), because the attention kernel must now dequantize Q8_0 KV entries on every decode step—adding fixed overhead regardless of context length. The crossover where KV Q8_0 becomes beneficial for Q2_K is around ctx=1400 tokens (3.75 tok/s default vs. 4.00 tok/s with KV Q8_0). For Q3_K_M (-11% cliff) and Q4_K_M (-12% cliff), KV Q8_0 roughly halves the degradation (-5%) with only a $\sim$1% throughput penalty at ctx=256—a favorable trade for applications targeting ctx≥1024. **Recommendation**: enable KV Q8_0 when ctx≥1400 with Q2_K, or when using Q3_K_M/Q4_K_M at ctx≥1024 for modest but consistent improvement.

Speculative decoding with a draft model. Meta's Llama 3.2 1B Instruct provides a natural draft model for speculative decoding [16] with the Llama 3.2 3B verifier. Combined GGUF size (Q4_K_M 3B + 1B) is ≈2.7 GB, fitting within the Tensor G1's 6 GB budget. At the memory-bandwidth-saturated decode regime observed here, CPU speculative decoding may yield 1.2–1.8× speedup with a suitable calibration corpus for the draft model.

P-core CPU affinity. llama.cpp supports `--cpu-mask` to pin inference threads to specific cores. On the Tensor G1, the correct performance-core mask is `0xF0` (cores 4–7: 2×Cortex-X1 at 2.8 GHz + 2×Cortex-A76 at 2.25 GHz); mask `0x0F` targets the little cores (cores 0–3, Cortex-A55 at 1.8 GHz) and reduces throughput by -25% for Q2_K. Measuring `0xF0` vs. the default scheduler is an immediately tractable experiment; community data on similar SoCs suggests 10–25% improvement.

Hardware performance counter validation. The mechanistic analysis in this paper is based on static instruction-count analysis of llama.cpp NEON/AVX2 source code. Direct validation via ARM Performance Monitor Unit (PMU) counters—specifically L2 cache miss rate, NEON utilization, and stall cycles per token—would confirm whether the Q6_K throughput penalty is dispatch-bound (many short vector operations) or bandwidth-bound (frequent L2 misses from larger weight tables). Android's `simpleperf` tool can expose ARM PMU counters on non-rooted devices for profiling on the Pixel 6a.

Broader quality evaluation. All six benchmarks in this study (BoolQ, ARC-Easy, ARC-Challenge, HellaSwag, MMLU, TruthfulQA) are classification or multiple-choice tasks; the model selects among a fixed answer set. Extending evaluation to generative tasks—GSM8K (grade-school math, chain-of-thought reasoning) and HumanEval (Python code generation and execution)—would test whether the Q2_K HellaSwag collapse (near-random on MCQ) extends to open-ended generation, where degenerate output modes may be harder to detect. Based on the observed Q2_K TruthfulQA failure (50% on yes/no questions = chance) and HellaSwag collapse (19%, below chance), we hypothesize that Q2_K will show severe degradation on GSM8K and HumanEval as well [18]. Preliminary on-device evaluation was attempted but revealed a binary interaction issue between `llama-completion`'s interactive mode and file-based prompt input; results from this attempt are excluded. Fixing the evaluation pipeline and running 50 GSM8K + 50 HumanEval questions per variant is the most impactful remaining quality experiment.

VII. Conclusion

We have presented a rigorous, multi-factor benchmark of seven GGUF quantization variants of Llama 3.2 3B Instruct on a Google Pixel 6a (Tensor G1, 6 GB LPDDR5), yielding 700+ valid measurements across throughput, context-scaling, and quality benchmarks spanning four context lengths (256–2048 tokens) and a thread-count scaling study.

Our primary empirical findings are:

1) **Non-monotonic throughput**: Q6_K (6-bit, 2.7 GB) delivers 3.52 tokens/s—slower than Q4_K_M (4.80 tokens/s) and Q8_0 (4.54 tokens/s) due to kernel dequantization overhead on ARM NEON.
2) **Context sensitivity is non-monotonic with bit-width**: Q3_K_M is the most context-stable variant ($< \pm 5\%$ across 256–2048 tokens). Q4_K_M also shows low degradation (-7%). Counterintuitively, Q2_K—the fastest at short context—is the *most* context-sensitive, losing -48% decode TPS from ctx=256 to 2048 (cliff onset at ctx=512, matching $L2/1024$ exactly). Q5_K_M (-26%) and Q8_0 (-18%) show substantial degradation. This non-monotonic pattern mirrors the throughput ordering anomaly and is explained by the relative KV-cache pressure on each variant's weight-tile footprint.
3) **Thread-count sweet spot**: 4 threads is optimal on the Tensor G1's heterogeneous cluster; 8 threads degrades throughput by 20% due to little-core load imbalance.
4) **BoolQ accuracy non-monotonicity**: On-device BoolQ reading comprehension accuracy does not monotonically improve with bit-width (65–74%): Q4_K_S (4-bit, small) achieves the highest score at 74%, Q6_K (6-bit) scores lowest at 65% despite having more bits per weight. ARC-Easy factual accuracy ranges 76–82% (Q4_K_M highest, Q2_K lowest), non-monotonic with bit-width and narrower than BoolQ variance. Q2_K exhibits instruction-following collapse on HellaSwag MCQ tasks (19%), despite 69% on BoolQ, revealing a task-format-specific failure mode. Q6_K is strictly dominated on both BoolQ accuracy and throughput, confirming it as the weakest deployment choice.
5) **Q4_K_S leads on accuracy; Q4_K_M recommended as default**: Q4_K_S (74%, 4.8 tokens/s) achieves the highest BoolQ accuracy of any variant. Q4_K_M (72%, 4.80 tokens/s) offers the most context-stable operating point and is the recommended default. Q2_K (69%, 5.11 tokens/s) is the speed-dominant choice for throughput-critical deployments, subject to the HellaSwag instruction-following collapse caveat. Q8_0 is dominated by Q4_K_M on both accuracy and throughput axes. Q4_K_M (2.0 GB) offers the best overall balance for 6 GB deployment.

These findings provide actionable guidance: use Q4_K_S when task accuracy is paramount (highest BoolQ at 74%); use Q4_K_M as the default (context-stable, 72% BoolQ, best overall balance); use Q2_K for latency-constrained or RAM-limited deployments (fastest at 5.11 tok/s, 69% BoolQ, but unsafe for structured MCQ tasks); use 4 threads on big.LITTLE ARM; and avoid Q6_K on CPU backends entirely—it is strictly dominated on both BoolQ accuracy and throughput, and catastrophically slow at ctx=2048. As on-device AI continues to proliferate, hardware-specific benchmarks of this kind are essential for evidence-based quantization selection.

References

[1] Meta AI, "Llama 3.2: Lightweight and Vision Language Models," Meta AI Blog, September 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[2] G. Gerganov et al., "llama.cpp: Efficient LLM inference in C/C++," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/llama.cpp

[3] G. Gerganov, "GGUF: A File Format for Language Model Weights," GitHub, llama.cpp project, 2023. [Online]. Available: https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *Proc. ICLR*, 2023.

[5] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proc. MLSys*, 2024.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. ICML*, 2023.

[7] T. Chen et al., "MLC-LLM: Universal LLM Deployment Engine with ML Compilation," arXiv preprint arXiv:2404.01373, 2024.

[8] Qualcomm Technologies, Inc., "On-Device AI: Large Language Models on Snapdragon," Qualcomm AI Research Technical Report, 2024. [Online]. Available: https://www.qualcomm.com/research/artificial-intelligence/ai-research

[9] R. Yi et al., "EdgeMoE: Fast On-Device Inference of MoE-Based Large Language Models," arXiv preprint arXiv:2308.14352, 2023.

[10] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. SOSP*, 2024.

[11] Meta AI, "ExecuTorch: On-Device AI across Mobile, Embedded, and Edge for PyTorch," GitHub Repository, 2024. [Online]. Available: https://github.com/pytorch/executorch

[12] AI@Meta, "Llama 3 Model Card," GitHub, 2024. [Online]. Available: https://github.com/meta-llama/llama3

[13] G. Gerganov, "GGML: Tensor Library for Machine Learning," GitHub Repository, 2023–2025. [Online]. Available: https://github.com/ggerganov/ggml

[14] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," in *Proc. NAACL-HLT*, 2019.

[15] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," arXiv preprint arXiv:1803.05457, 2018.

[16] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023. arXiv:2211.17192.

[17] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," arXiv preprint arXiv:2401.18079, 2024.

[18] Y. Tang et al., "Evaluating Quantized Large Language Models," arXiv preprint arXiv:2402.18158, 2024.

[19] S. Laskaridis, A. Mehrotra, and I. Venieris, "On the Energy Consumption of Large Language Models at the Edge," arXiv preprint arXiv:2403.06928, Samsung AI Research, 2024.

[20] S. Laskaridis, A. Antonini, A. Katsianos, N. D. Lane, and S. I. Venieris, "MELTing Point: Mobile Evaluation of Language Transformers," in *Proc. MobiSys*, 2024.

[21] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression," in *Proc. ICLR*, 2024.

[22] T. Dettmers and L. Zettlemoyer, "The Case for 4-bit Precision: k-bit Inference Scaling Laws," in *Proc. ICML*, 2023.

[23] C. Liu, T. Shi, M. Xia, K. Shao, J. Chen, and S. Luo, "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases," arXiv preprint arXiv:2402.14905, 2024.

[24] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently Scaling Transformer Inference," in *Proc. MLSys*, 2023.

[25] S. Biderman et al., "Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling," in *Proc. ICML*, 2023.

[26] E. Georganas, S. Avancha, K. Banerjee, D. Kalamkar, G. Henry, H. Pabst, and A. Heinecke, "Anatomy of High-Performance Deep Learning Convolutions on SIMD Architectures," in *Proc. SC*, 2018.
[27] R. Y. Aminabadi et al., "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," in *Proc. SC*, 2022.
[28] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Carpenter, S. Ghodrati, V. Bhatt, A. Mishra, M. Rastegari, and M. Farajtabar, "LLM in a Flash: Efficient Large Language Model Inference with Limited Memory," in *Proc. CVPR*, 2024.
[29] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in *Proc. NeurIPS*, 2022.
[30] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks," arXiv preprint arXiv:2402.04396, 2024.
[31] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.
[32] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *Proc. ICLR*, 2024.
[33] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.
[34] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
[35] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?" in *Proc. ACL*, 2019.
[36] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring Massive Multitask Language Understanding," in *Proc. ICLR*, 2021.
[37] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," in *Proc. ACL*, 2022.
[38] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," in *Proc. ICLR*, 2017.
[39] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez, and I. Stoica, "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," arXiv preprint arXiv:2403.04132, 2024.

APPENDIX

A. Build Configuration

The llama.cpp library was compiled from commit `b1-1a29907` using the Android NDK version 29.0.14206865. Compilation flags:

```
-O3 -march=armv8.2-a+dotprod+fp16
-DGGML_USE_CPU_HBM=OFF
-DLLAMA_BUILD_TESTS=OFF
```

The resulting `llama-cli` binary was pushed to `/data/local/tmp/` on the device and executed via ADB shell. The Android app was compiled with Android NDK 29.0.14206865, targeting arm64-v8a, using CMake 4.2.3 and Gradle 8.x. All source code, benchmark scripts, and raw results are available in the project repository.

B. Invocation Template

Each trial was executed with the following command template:

```
./llama-cli \
  -m <model_path>.gguf \
  -p "<system_prompt><user_prompt>" \
  -n 128 \
  --ctx-size <context_length> \
  --seed <trial_seed> \
  -t <thread_count> \
  --log-format json
```

C. Statistical Methodology

Means and standard deviations in Table I are computed across 15 samples per variant per context (5 trials $\times$ 3 prompt types). Warmup runs (2 per configuration) are excluded. Outlier rejection was not applied; all completed trials within the timeout threshold are included in analysis.

D. BoolQ (100 questions)

BoolQ questions are drawn from the SuperGLUE validation split [14]. Each example consists of a passage (avg. $\sim$100 words) and a yes/no question. The prompt template is:

```
Passage: {passage}
Question: {question}
Answer with only yes or no:
```

Responses are scored by extracting the first occurrence of the words *yes* or *no* (case-insensitive) from the model output, within 400 characters following the instruction marker.

E. ARC-Easy (100 questions)

ARC-Easy questions are drawn from the AI2 ARC Easy test split [15]. Each example presents a 4-choice multiple-choice question. The prompt template is:

```
Question: {question}
Choices: A) {A} B) {B} C) {C} D) {D}
Answer with only A, B, C, or D:
```

Scoring uses exact-match on the predicted letter (A/B/C/D). The `model_output` field in stored JSON records the extracted letter only (e.g., "A"), not the raw llama-completion output, ensuring stored results are directly verifiable and reproducible. Verified accuracy across all seven variants: Q2_K 76%, Q3_K_M 78%, Q4_K_S 81%, Q4_K_M 82%, Q5_K_M 81%, Q6_K 79%, Q8_0 80%.